

Assignment 1 Dart Basics Script

Name: Parth Sahani

Cohort: Elon Musk

Roll Number: 150096724135

Github Fork Link:

https://github.com/Indianworldruler/parth_assignments/

Please do check the branches for all the submitted assignments

Explanation at the last of the screenshots

PLEASE TURN OVER

Screenshots:

- (base) parthsahani@Parths-MacBook-Air oop_tasks % dart run task1_class.dart
Book Title: Harry Potter
Author: J.K. Rowling
- (base) parthsahani@Parths-MacBook-Air oop_tasks % dart run task2_constructor.dart
Book Title: The Alchemist
Author: Paulo Coelho
- (base) parthsahani@Parths-MacBook-Air oop_tasks % dart run task3_methods.dart
Book Title: Atomic Habits
Author: James Clear
Atomic Habits is now open.
Atomic Habits has been closed.
- (base) parthsahani@Parths-MacBook-Air oop_tasks % dart run task4_inheritance.dart
Book Title: Clean Code
Author: Robert C. Martin
File Size: 5.2 MB
- ⊗ (base) parthsahani@Parths-MacBook-Air oop_tasks % task5_mixin.dart
zsh: command not found: task5_mixin.dart
- (base) parthsahani@Parths-MacBook-Air oop_tasks % dart run task5_mixin.dart
Book Title: The Kite Runner
Author: Khaled Hosseini
Book Action: Book borrowed
- (base) parthsahani@Parths-MacBook-Air oop_tasks % dart run task6_interface.dart
Book Type: Printed Book
Book Title: 1984
Author: George Orwell
- (base) parthsahani@Parths-MacBook-Air oop_tasks % dart run task7_list.dart
Book Title: Harry Potter
Author: J.K. Rowling

Book Title: The Alchemist
Author: Paulo Coelho

Book Title: Atomic Habits
Author: James Clear

Book Title: The Hobbit
Author: J.R.R. Tolkien

- (base) parthsahani@Parths-MacBook-Air oop_tasks % dart run task8_map.dart
Author: J.R.R. Tolkien
Book: Harry Potter
Author: J.K. Rowling

Book: The Alchemist
Author: Paulo Coelho

Book: Atomic Habits
Author: James Clear

Book: The Hobbit
Author: J.R.R. Tolkien

```

• (base) parthsahani@Parths-MacBook-Air oop_tasks % dart run task9_set.dart
Unique Books:
Harry Potter
The Alchemist
Atomic Habits
The Hobbit
Clean Code
Total Books: 5

• (base) parthsahani@Parths-MacBook-Air oop_tasks % dart run task10_library_system.dart
===== MY LIBRARY SYSTEM =====

===== BOOK DETAILS =====
Book: Harry Potter
Author: J.K. Rowling
Status: Available
Harry Potter is now open.
Harry Potter has been closed.

===== BORROW BOOK =====
Harry Potter has been borrowed.
Book: Harry Potter
Author: J.K. Rowling
Status: Borrowed

===== RETURN BOOK =====
Harry Potter has been returned.
Book: Harry Potter
Author: J.K. Rowling
Status: Available

===== EBOOK =====
Book: Clean Code
Author: Robert C. Martin
Status: Available
File Size: 5.2 MB

===== MIXIN =====
Book: The Kite Runner
Author: Khaled Hosseini
Status: Available
Book Action: Book borrowed

===== INTERFACE =====
Type: Printed Book
Title: 1984
Author: George Orwell
-----
Type: Digital Book
Title: The Great Gatsby
Author: F. Scott Fitzgerald

```

===== LIBRARY LIST =====

Book: Harry Potter
Author: J.K. Rowling
Status: Available

Book: The Alchemist
Author: Paulo Coelho
Status: Available

Book: Atomic Habits
Author: James Clear
Status: Borrowed

Book: Clean Code
Author: Robert C. Martin
Status: Available

Book: The Kite Runner
Author: Khaled Hosseini
Status: Available

===== BOOK MAP =====

Book: Harry Potter
Author: J.K. Rowling

Book: The Alchemist
Author: Paulo Coelho

Book: Atomic Habits
Author: James Clear

Book: Clean Code
Author: Robert C. Martin

Book: The Kite Runner
Author: Khaled Hosseini

===== UNIQUE BOOKS =====

Harry Potter
The Alchemist
Atomic Habits
Clean Code
The Kite Runner
Total Books: 5

===== FINAL LIBRARY STATUS =====

Harry Potter : Available
The Alchemist : Available
Atomic Habits : Borrowed
Clean Code : Available
The Kite Runner : Available

Library system completed.

What I Learned From The Library System Assignment

1. Understanding Class

- First I learned how a class is used to make a structure for objects.
- In my library system I used the Book class.
- The class contains book information like title author and isBorrowed.
- This made the code more organised because all book related things are kept inside the Book class.
- I understood that one class can be used to create many different book objects.

For example I created books like Harry Potter, The Alchemist and Atomic Habits. They all use the same Book class but their values are different.

I also learned that class is like a blueprint. The blueprint tells what information and actions the object will have. The actual book object is created from that blueprint.

2. Constructor

- I learned how a constructor is used when creating an object.
- In my code I used:

```
Book(this.title, this.author, this.isBorrowed);
```

- This allows me to give the book details when creating the object.
- Instead of assigning every value separately I can directly give the values while creating the book.

For example:

```
Book book1 = Book("Harry Potter", "J.K. Rowling", false);
```

This creates a new book with its title author and borrowing status.

I understood that constructor makes object creation easier. It also saves some extra lines of code.

3. Methods

- I learned that methods are functions written inside a class.
- They are used to perform actions related to the object.
- In my Book class I created methods like `displayBook()` `openBook()` `closeBook()` `borrowBook()` and `returnBook()`.

The `displayBook()` method shows the book details. The `openBook()` method shows that the book is opened. The `borrowBook()` method changes the borrowing status.

This helped me understand that a class can contain both data and actions. The variables store information while methods perform work on that information.

4. Inheritance

- I learned how inheritance allows one class to use the properties and methods of another class.
- In my library system `EBook` extends `Book`.

So `EBook` automatically gets the book information and methods from `Book`.

I also added `fileSize` in `EBook` because an electronic book can have a file size. This showed me that the child class can have something extra while still using the features of the parent class.

For example:

```
EBook ebook = EBook("Clean Code", "Robert C. Martin",  
false, 5.2);
```

The EBook can use `displayBook()` from Book. It can also use its own `displayFileSize()` method.

I understood inheritance as a way of reusing existing code instead of writing the same code again.

5. Mixin

- I learned about mixin which is used to add extra functionality to a class.
- I created a BookLogger mixin.
- It contains the `logBookAction()` method.
- Then I used it with LibraryBook.

This means LibraryBook can use the normal Book features plus the logging feature.

For example:

```
libraryBook.logBookAction("Book borrowed");
```

It prints the action performed on the book.

At first mixin was a little confusing because it is different from normal inheritance. Later I understood that it is mainly useful for adding some common functionality to a class.

6. Interface

- I also learned how interface works in Dart.
- I created an abstract class called LibraryItem.
- It contains the `displayItem()` method.
- Then PrintedBook and DigitalBook implement this interface.

Both classes have to provide their own version of `displayItem()`.

This helped me understand that different classes can follow the same structure while doing the work in their own way.

A `PrintedBook` displays "Printed Book" while `DigitalBook` displays "Digital Book". Both follow the same `LibraryItem` structure.

7. List

- I learned how to store multiple books using a `List`.
- I created:

```
List<Book> library = [...]
```

- I added different book objects into this list.
- I also used a `for` loop to display every book.

This was useful because a real library will have many books. Creating separate variables for every book would make the program bigger. A list makes it easier to store and process many books together.

I also learned that objects created from a class can be stored inside a list.

8. Map

- I learned how `Map` stores data using a key and a value.
- In my example the book title is the key while the author is the value.

For example:

```
"Harry Potter": "J.K. Rowling"
```

This makes it easy to find an author using the book title.

I also used `forEach()` to display all the books and authors. This helped me understand how `Map` can be used when two related pieces of information need to be connected.

9. Set

- I learned that a Set stores unique values.
- In my library system I used a Set for book names.
- I tried adding "Harry Potter" twice.
- The duplicate value was not added again.

This showed me the main difference between a List and Set. A List can contain duplicate values while a Set keeps only unique values.

I also used `.length` to find the total number of unique books.

How I Combined All The Concepts

The main learning from this assignment was not only learning each concept separately. The important part was understanding how all these concepts can work together in one program.

I started with the Book class because the whole library system is based around books. First I added the variables for title author and borrowing status. Then I created a constructor so that I can create different book objects easily.

After that I added methods for displaying opening closing borrowing and returning books. This made the Book class more useful because the books could now perform actions instead of only storing information.

Then I added inheritance with the EBook class. EBook uses the Book class but also has its own file size. After this I added a mixin for logging book actions. This showed me how extra features can be added without changing the main Book class.

Then I used an interface for PrintedBook and DigitalBook. Both classes follow the same LibraryItem structure but display different information.

After the object oriented concepts were completed I used collections. I created a List to store the books in the library. I used a Map to connect book titles with authors. I used a Set to store unique book names.

The overall flow became something like:

Class → Constructor → Methods → Inheritance → Mixin →
Interface → List → Map → Set → Main Library System

I understood that these concepts are not separate things which can only be used individually. They can be combined to create a proper working program.

Creating The Main Library System

- In the `main()` function I created different Book objects.
- Then I called their methods.
- I tested borrowing and returning books.
- I created an EBook object to test inheritance.
- I created a LibraryBook object to test the mixin.
- I created PrintedBook and DigitalBook objects for the interface.
- After that I created the List Map and Set.
- Finally I displayed the complete library status.

I learned that `main()` is where I can run and test the different parts of the program.

The library system is simple but it gives a proper example of how the concepts can be connected together. I also learned that the order of writing the code is important. Classes and methods should be created first so they can be used later inside `main()`.

What I Understood From The Assignment

- Class gives a structure to objects.
- Constructor helps create objects with values.
- Methods perform actions.
- Inheritance helps reuse code.
- Mixin adds extra functionality.
- Interface gives a common structure to different classes.

- List stores multiple values in order.
- Map stores key and value pairs.
- Set stores unique values.
- `main()` is used to run the complete program.

The biggest thing I understood is that object oriented programming is useful for making a bigger program in a clean way. Instead of putting everything inside `main()` I can divide the program into different classes and features.

I also understood why we use different collection types. If I need all books I can use a List. If I need to connect a book with its author I can use a Map. If I only need unique book names I can use a Set.

Problems I Faced While Writing The Assignment

- One basic problem I faced was remembering the syntax of constructors.
- Sometimes I was confused about where the constructor should be written inside the class.
- In inheritance I had some confusion about using `super`.
- I also had to understand how the child class gets methods from the parent class.
- Mixin was a little confusing in the beginning because it is written differently from inheritance.
- Interface was also slightly confusing because the class has to implement the required method.
- I faced some small syntax errors like missing brackets commas or semicolons.
- While working with List Map and Set I sometimes mixed their syntax.
- Another problem was managing many classes in the same file.
- I had to check the code section by section instead of trying to write the complete program at one time.
- Sometimes the output was not what I expected because I forgot to update or check the book borrowing status.

